AI-BASED USER AND ENTITY BEHAVIOUR ANALYTICS SYSTEM

AN IN-HOUSE PROJECT REPORT

Submitted by

AMAAN SAIFY (2023BCSE07AED025)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Under the Supervision of

Dr. Rathnakar Achary



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

OCTOBER - 2026

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

CERTIFICATE

This is to certify that the In-house Project work entitled "AI-BASED USER AND ENTITY BEHAVIOUR ANALYTICS SYSTEM" is the bonafide work done by **AMAAN SAIFY (2023BCSE07AED025)** submitted in partial fulfilment of the requirements for the award of the degree Bachelor of Technology in **Computer Science and Engineering (Cyber Security)** during the year 2026-27.

Dr. Rathnakar Achary
Associate Professor
Dept. of Computer Science and Engineering

DECLARATION

This is to declare that the report titled "AI-Based User and Entity Behaviour Analytics System" has been made for the partial fulfilment of the Programme - Bachelor of Technology in **Computer Science and Engineering (Cyber Security)**, under the guidance of **Dr. Rathnakar Achary**. I confirm that this report truly represents my work undertaken as a part of my **In-house Project** work. This work is not a replication of work done previously by any other person. I also confirm that the contents of the report and the views contained therein have been discussed and deliberated with the Mentor.

NAME	REGESTRATION NO.	SIGNATURE
AMAAN SAIFY	2023BCSE07AED025	

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people that made it possible, whose constant guidance and encouragement crown all the efforts successfully.

I am very thankful to my supervisor **Dr. Rathnakar Achary** for his sustained inspiring guidance and cooperation throughout the process of this project. His wise counsel and valuable suggestions are invaluable.

I would like to thank **Dr. K. Ramalakshmi**, Head of the Department, Computer Science and Engineering (Cyber Security), and **Dr. Vullikanti Vivek**, Deputy Director / ASAC, for their encouragement and cooperation at various levels of the project.

I avail this chance to express my deep sense of gratitude and hearty thanks to the Management of Alliance University, for providing world class infrastructure, congenial atmosphere, and encouragement.

I express my deep sense of gratitude to the teaching and non-teaching staff of our department who stood with me during the project and helped me to make it a successful venture.

I place highest regards to my parents, my friends and well-wishers who helped plenty in making the report of this project.

Amaan Saify (2023BCSE07AED025)

ABSTRACT

Most data breaches today do not begin with a software exploit. They begin with a stolen password or with an employee who already has access. Signature-based tools such as antivirus software and intrusion detection systems look for known bad patterns, so a login with valid credentials or a slow insider copying files looks normal to them. User and Entity Behaviour Analytics (UEBA) addresses this by learning how each user and device normally behaves and flagging changes from that behaviour.

This project develops a Python-based UEBA system. It captures four kinds of logs (authentication, system access, network and resource usage), turns each user's and device's day into 44 and 19 behavioural features respectively, and learns a personal baseline and a department peer baseline for every entity. Deviations are scored by three independent layers: robust statistics, an unsupervised machine-learning ensemble (Isolation Forest, Local Outlier Factor and an autoencoder) and 20 threat rules mapped to MITRE ATT&CK. The scores are fused into a 0-100 risk score weighted by asset criticality, and every alert carries plain-English reasons. Baselines adapt over time, but suspicious days are quarantined so that an attacker cannot teach the system to accept an attack. Analysts can mark alerts as benign or confirmed through a Streamlit dashboard, and this feedback updates the profiles.

The system was evaluated on 60 days of simulated logs from a 48-person organisation (about 205,000 events, 108 monitored entities) containing 11 planted attacks and 4 harmless but unusual events. All 11 attacks were detected on the day they started, with precision, recall and F1-score of 0.964 and one false alert in 30 days. Across five differently generated organisations the mean F1-score was 0.951, and with all rules switched off the behavioural layers alone still reached an F1-score of 0.857.

Keywords: UEBA, insider threat, anomaly detection, Isolation Forest, autoencoder, risk scoring, explainable alerts, SOC.

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation	Meaning
AE	Autoencoder
AUC	Area Under the Curve
CERT	Computer Emergency Response Team (CMU insider-threat dataset)
CISO	Chief Information Security Officer
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EDR	Endpoint Detection and Response
FN / FP	False Negative / False Positive
FPR	False Positive Rate
GCN / GNN	Graph Convolutional Network / Graph Neural Network
IF	Isolation Forest
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
MAD	Median Absolute Deviation
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
ML	Machine Learning
P95	95th percentile
PR-AUC	Area under the Precision-Recall curve
ROC	Receiver Operating Characteristic
SIEM	Security Information and Event Management
SOC	Security Operations Centre
TN / TP	True Negative / True Positive
UEBA	User and Entity Behaviour Analytics
VPN	Virtual Private Network
z	Robust z-score (deviation measured in units of typical spread)

CONTENTS

Page No

Title Page ..... **i**

Certificate ..... **ii**

Declaration ..... **iii**

Acknowledgement ..... **iv**

Abstract ..... **v**

List of Symbols & Abbreviations ..... **vi**

1. Introduction **1**

2. Literature Review **2**

3. Identification of the Problem / Scope of the Project **10**

4. Proposed Idea / Solution **12**

5. Objectives **13**

6. Process / Methodology **14**

6.1 Identify the needs of customer and users through Design Thinking Methods .....

6.2 Translate the needs of customer into Project / Product.....

6.3 Specification of the Project / Product .....

6.4 Designing the solution for the customer needs .....

6.5 Process / Flow / Design Diagram.....

7. Requirements **22**

7.1 Hardware Requirements.....

7.2 Software Requirements .....

7.3 Datasets .....

7.4 Technologies and Tools .....

8. Novelty of the Proposed Idea / Project **24**

9. Experimentation **25**

10. Results and Discussion **27**

11. Conclusion and Future Directions **32**

References **33**

Source Code **35**

Submission Page of the Paper **38**

Plagiarism Report **39**

1. INTRODUCTION

Organisations depend on user accounts to control who can do what. That same dependence makes accounts the easiest way in for an attacker. The 2025 Verizon Data Breach Investigations Report found that credential abuse was the most common initial access vector, involved in 22% of breaches, and that the human element played a part in about 60% of breaches [1]. Insiders are an equally serious problem: the 2025 Cost of Insider Risks report puts the average annual cost of insider incidents at US$17.4 million per organisation, with 81 days needed on average to contain an incident [2].

Traditional security tools are good at recognising known bad things: a malware signature, a blacklisted IP address, a known exploit. They struggle when the attacker uses a real username and password, or when the person causing harm is an employee who already has access. In these cases every log line can look legitimate. What gives the attack away is behaviour: a salesperson logging in at 3 a.m. from another country, an engineer reading ninety files instead of the usual fifteen, a laptop uploading gigabytes to a file-sharing site it has never used before.

User and Entity Behaviour Analytics (UEBA) is the class of security analytics built for this situation. A UEBA system learns a profile of normal behaviour for every user and every entity (devices, servers, applications) and raises an alert when current behaviour deviates from that profile [3]. Because it does not need a signature, it can catch stolen-credential attacks, insider activity and previously unseen techniques. The practical difficulty is false alarms. Human behaviour is irregular, and security analysts already report being flooded with alerts that turn out to be harmless [4]. A useful UEBA system therefore has to be accurate, quiet and able to explain itself.

This project builds such a system in Python. It captures authentication, system-access, network and resource-usage logs, builds behavioural profiles for users and devices, identifies deviations with three complementary detection layers, and produces prioritised, explained, risk-based alerts in an analyst dashboard. Each alert is tagged with techniques from the MITRE ATT&CK framework [5] so that analysts can relate it to known attacker behaviour.

1.1 Organisation of the Report

Chapter 2 reviews recent research on UEBA and insider-threat detection. Chapter 3 defines the problem and the scope. Chapters 4 and 5 present the proposed solution and its objectives. Chapter 6 describes the methodology, starting from user needs identified through design thinking and ending with the system design and diagrams. Chapter 7 lists the requirements and Chapter 8 explains what is new about the project. Chapters 9 and 10 describe the experiments and discuss the results, and Chapter 11 concludes with future directions.

2. LITERATURE REVIEW

This chapter reviews nine primary research papers published from 2022 onwards on UEBA, insider-threat detection, lateral-movement detection and alert triage. Each paper is summarised with its abstract, accomplishments and limitations, and its relevance (scope) to this project. Section 2.10 compares the papers and identifies the research gap.

2.1 Paper 1: A User and Entity Behavior Analytics Log Data Set for Anomaly Detection in Cloud Computing [3]

Published in: IEEE BigData 2022.

Abstract:

The authors note that compromised accounts let attackers act through legitimate operations, which signature-based tools cannot recognise, and that very few real log data sets exist for testing UEBA methods. They publish CLUE-LDS, a real log data set from a cloud file-sharing platform covering more than 5,000 users over more than five years. Because real data has no labelled attacks, they propose injecting account-hijacking anomalies by swapping the identities of two users with similar behaviour. They also present an adaptive anomaly detector that compares each user-day's event counts with a queue of earlier days and retrains whenever behaviour changes.

Accomplishments and Limitations:

The paper provides a rare public, real-world UEBA data set and a reproducible attack-injection method. With a model size of 30 days and a threshold of 0.6, the adaptive detector reached a true positive rate of 65% at a false positive rate of 4.6% (fewer than two false alarms per day), spending 6.8% of days in retraining. The authors also show that real user behaviour changes abruptly and often, and they warn that false alarm rates measured on synthetic data may be unrealistically low. The limitations are that detection uses only event-type frequencies from a single log source, the attacks are simulated, and there is no explanation or risk prioritisation of alerts.

Scope:

This work supports two design decisions of the project: an adaptive baseline that keeps learning, and an honest statement that results on synthetic data are likely optimistic. It also shows that a detection rate near 65% at a 4.6% false positive rate is a realistic level on real data, which frames how our synthetic results should be read.

2.2 Paper 2: Multi-homed Abnormal Behavior Detection Algorithm Based on Fuzzy Particle Swarm Cluster in User and Entity Behavior Analytics [6]

Published in: Scientific Reports 2022.

Abstract:

The paper treats UEBA as anomaly detection for both internal threat analysis and external intrusion detection, and models the behaviour of entities as well as users. It proposes MAD-FPC, which groups entity behaviour with fuzzy clustering whose cluster centres are searched with particle swarm optimisation. A behaviour frequency-inverse entities frequency (BF-IEF) measure, similar to TF-IDF in text mining, gives more weight to rare behaviour when comparing entities. A point is judged abnormal by looking at its membership in several clusters at once (multi-homed attribution) instead of a single cluster.

Accomplishments and Limitations:

On simulated entity and user connection behaviour built from the NSL-KDD data set, the method reported an accuracy of 0.92, precision of 0.96, recall of 0.90 and F1-score of 0.93, better than the traditional detectors it was compared with. Its strength is entity-aware, unsupervised detection with a sensible rarity weighting. Its limitations are the evaluation data (NSL-KDD is an old network intrusion data set, not a behaviour log), a static model that does not adapt over time, and no explanation or risk scoring of the anomalies it finds.

Scope:

The paper supports profiling devices and other entities, not only users, which this project does with 19 device features. The idea of weighting rare behaviour more heavily is reflected in our novelty features (new hosts, new domains, new countries).

2.3 Paper 3: Cybersecurity Threat Detection Based on a UEBA Framework Using Deep Autoencoders [7]

Published in: AIMS Mathematics 2025.

Abstract:

The authors build a UEBA anomaly-detection framework for a financial institution using deep autoencoders, with Doc2Vec embeddings so that both numeric and text log fields can be used. The autoencoder is trained on data assumed to be mostly normal, and a record is flagged when it cannot be reconstructed well. Feature-level reconstruction residuals are used to explain which fields made a record anomalous. The paper also contributes a proof that two common definitions of fully connected neural networks are equivalent.

Accomplishments and Limitations:

The framework handles mixed numeric and textual logs and gives feature-level explanations, which the authors argue makes it practical for security teams. The real data set had no anomaly labels, so precision, recall and F1-score could not be computed; the few real attack events available were detected almost trivially, so the authors evaluated sensitivity with synthetic anomalies of increasing intensity.

Limitations are reliance on a single model type, the absence of standard detection metrics, and no fused or prioritised risk score.

Scope:

This paper is the closest published example of an explainable UEBA system and supports the autoencoder used in our ML ensemble. Our project extends the idea by combining the autoencoder with two other unsupervised models, statistics and rules, and by turning deviations into plain-English reasons inside a single risk score.

2.4 Paper 4: Hunting for Insider Threats Using LSTM-Based Anomaly Detection [8]

Published in: IEEE Transactions on Dependable and Secure Computing 2023.

Abstract:

Insiders mix the steps of an attack with their normal work, which makes the attack hard to see in logs. The authors propose LADOHD, an anomaly-detection framework based on Long Short-Term Memory (LSTM) networks that learns the expected order of events on endpoints and flags event sequences that do not fit, even when an attack spreads over several days.

Accomplishments and Limitations:

On 38.9 million endpoint events collected from a commercial network of 30 computers over twenty days, containing a four-day insider attack, LADOHD achieved a true positive rate of 97.29% with a false positive rate of 0.38%, outperforming the anomaly-detection system already protecting that network. The authors also show that LSTMs predict variable-length event sequences more precisely than Hidden Markov Models. Limitations: it uses a single source of endpoint events, its alerts are not explained to analysts, it was tested on one organisation, and it needs a clean training period.

Scope:

The paper is the main example of deep sequence modelling for insider threats. It shows what order-aware models can achieve and is listed as future work for this project, which uses daily feature vectors because they are lighter and easier to explain.

2.5 Paper 5: A High Accuracy and Adaptive Anomaly Detection Model With Dual-Domain Graph Convolutional Network for Insider Threat Detection [9]

Published in: IEEE Transactions on Information Forensics and Security 2023.

Abstract:

The authors argue that most insider-threat methods treat each user's behaviour as an isolated sequence and ignore relationships between users, which leads to false positives. DD-GCN converts user features and relationships into heterogeneous graphs, balances feature similarity against existing links with a

weighted similarity mechanism, and applies graph convolution in two domains (feature and topology) with constraints that keep the learned representations consistent but distinct.

Accomplishments and Limitations:

On the CERT r4.2 and r6.2 insider-threat data sets the model reached accuracies of 98.21% and 98.65% respectively, a clear margin over earlier methods. Its limitations are that it is semi-supervised and relies on a portion of labelled users (the experiments vary the proportion of labels), it profiles users only, it is evaluated only on the synthetic CERT data, and it gives no explanation or prioritised risk score.

Scope:

The paper shows the value of comparing a user with related users. This project captures a simpler, explainable version of the same idea through department peer-group baselines, and it needs no labels.

2.6 Paper 6: LAN: Learning Adaptive Neighbors for Real-Time Insider Threat Detection [10]

Published in: IEEE Transactions on Information Forensics and Security 2024.

Abstract:

Earlier insider-threat work usually flags a suspicious user or a suspicious day after the fact. LAN instead detects malicious activity in real time at the level of individual activities. It learns temporal dependencies within each user's activity sequence and relationships between activities across sequences using graph structure learning, and it uses a hybrid prediction loss that combines self-supervision from normal activities with supervision from the few abnormal ones to cope with extreme class imbalance.

Accomplishments and Limitations:

On CERT r4.2 and r5.2, LAN outperformed nine state-of-the-art baselines by at least 9.92% and 6.35% in AUC for real-time detection, and it also outperformed eight baselines in post-hoc detection. Limitations: its loss uses labels from abnormal activities, it profiles users only, it was evaluated only on CERT data, and its deep model does not explain its decisions.

Scope:

LAN shows where the field is heading (real-time, activity-level detection). This project works at the day level, which is simpler and easier to explain, and lists streaming detection as future work.

2.7 Paper 7: AUTH: An Adversarial Autoencoder Based Unsupervised Insider Threat Detection Scheme for Multisource Logs [11]

Published in: IEEE Transactions on Industrial Informatics 2024.

Abstract:

AUTH targets the stealth of insider behaviour and the heavy imbalance between normal and malicious data. It is fully unsupervised and uses both time features and event features from multiple log sources.

Its core model, TL-AAE, is an autoencoder whose encoder combines a temporal convolutional network and an LSTM, trained adversarially in the style of a generative adversarial network to make the latent space more regular.

Accomplishments and Limitations:

On a widely used public insider-threat data set, AUTH achieved an AUC of 0.932, 4.95% higher than the best baseline, and an equal error rate of 0.146, 12.57% lower than the best baseline. The ablation study shows that time features improve detection. Limitations: it profiles users only, its anomaly scores are not explained, it produces no risk ranking, and it assumes the training data is mostly benign without any protection against slow baseline poisoning.

Scope:

The paper supports two choices in this project: unsupervised learning, so that no attack labels are needed, and time-of-day features such as logins at unusual hours.

2.8 Paper 8: Euler: Detecting Network Lateral Movement via Scalable Temporal Link Prediction [12]

Published in: ACM Transactions on Privacy and Security 2023.

Abstract:

Lateral movement, where an attacker moves from one machine to another, is a key stage of advanced attacks. Euler represents network and authentication logs as a series of graph snapshots and treats lateral movement as the appearance of unlikely edges. It stacks a graph neural network on a recurrent sequence model and distributes the graph computation across machines so that it scales to very large logs.

Accomplishments and Limitations:

On the Los Alamos National Laboratory 2015 data set (58 days of real network activity, about 1.6 billion events, with red-team activity), a GCN-GRU Euler model reached an AUC of about 0.99 with a true positive rate of 89.65% and a false positive rate of 0.57%, and trained much faster than earlier models. However, its precision was only about 0.006, meaning almost every alert was a false alarm, because real attacks are extremely rare among benign connections. It models host-to-host links only and does not explain its alerts.

Scope:

Euler supports the lateral-movement scenario and the device profiling in this project. Its very low precision despite a high AUC is strong evidence for the project's focus on reducing false positives through corroboration between layers.

2.9 Paper 9: DEEPCASE: Semi-Supervised Contextual Analysis of Security Events [13]

Published in: IEEE Symposium on Security and Privacy 2022.

Abstract:

Security operators often need several related events to decide whether something is a real threat, and the volume of events forces them to ignore many. DeepCASE uses an attention-based neural network to learn the context (the events that came before) of each security event, clusters similar event sequences, and lets an operator label a few sequences per cluster so that the label is applied to the rest. The attention weights show which earlier events drove the decision.

Accomplishments and Limitations:

On a real data set of 10.5 million security events from 20 international organisations over five months, DeepCASE automatically filtered 86.72% of events and reduced the operators' manual workload by 90.53%, while underestimating the risk of a potential threat in less than 0.001% of cases. Limitations: it works on alerts that other detectors have already raised rather than on raw user behaviour, and it needs operator labelling.

Scope:

DeepCASE is strong evidence that context and explanation reduce analyst workload. It supports the project's explained alerts, its analyst feedback loop and its decision to rank alerts by risk instead of presenting them as a flat list.

2.10 Comparison and Research Gap

Table 2.1: Comparison of the reviewed papers

Paper	Data	Technique	Profiles	Explained	Risk score	Key result
[3] Landauer 2022	Real cloud logs	Adaptive similarity	Users	No	No	TPR 65%, FPR 4.6%
[6] Cui 2022	NSL-KDD	Fuzzy PSO clustering	Users + entities	No	No	F1 0.93
[7] Fuentes 2025	Real bank logs	Autoencoder + Doc2Vec	Users + entities	Yes	No	No labels for metrics
[8] LADOHD 2023	Real EDR, 30 PCs	LSTM	Endpoints	No	No	TPR 97.29%, FPR 0.38%
[9] DD-GCN 2023	CERT r4.2, r6.2	Graph conv. net	Users	No	No	Acc. 98.21%, 98.65%
[10] LAN 2024	CERT r4.2, r5.2	Graph + sequence	Users	No	No	+9.92% AUC
[11] AUTH 2024	Public insider data	Adversarial AE	Users	No	No	AUC 0.932
[12] Euler 2023	LANL, 1.6B events	Temporal GNN	Hosts	No	No	AUC 0.99, precision 0.006
[13] DeepCASE 2022	20 orgs, 10.5M events	Attention + clustering	Alerts	Yes	Partly	Workload -90.53%
This project	Simulated org + CERT adapter	Stats + IF/LOF/AE + rules	Users + devices	Yes	Yes	F1 0.964 (Ch. 10)

Recent work has pushed detection accuracy on public insider-threat benchmarks very high [9]-[11], but these systems profile users only, depend on deep models that analysts cannot question, and stop at an anomaly score without ranking or explaining it. Work that profiles entities either models hosts on their

own [12] or uses data that is not behaviour logs [6]. Explanation has begun to appear [7], [13], and adaptive baselines are clearly needed for real, changing behaviour [3], but these ideas have not been combined in one pipeline. None of the reviewed systems protects its baseline against an attacker who slowly makes malicious behaviour look normal, and none corroborates machine-learning anomalies with ATT&CK-mapped rules to cut false positives, even though [12] shows how badly precision suffers without such filtering.

The gap this project addresses is therefore one of integration and usability rather than a single new algorithm: a system that profiles users and devices together against personal and peer baselines, keeps those baselines adaptive but poisoning-resistant, requires agreement between statistical, machine-learning and rule-based evidence before alerting, and presents a single explained risk score to the analyst. Its main weakness relative to [9]-[11] is that it is evaluated on simulated logs rather than a large labelled benchmark, which is discussed in Chapter 10.

2.11 Any Other Sources

- **MITRE ATT&CK knowledge base** [5]: used to map every threat rule to a technique ID such as T1110 (Brute Force) or T1567 (Exfiltration Over Web Service).
- **Verizon Data Breach Investigations Report 2025** [1]: source of breach statistics on credential abuse and the human element.
- **Ponemon / DTEX Cost of Insider Risks 2025** [2]: source of insider-incident cost and containment-time figures.
- **CMU CERT Insider Threat Test Dataset** [14]: the standard public benchmark for insider-threat research; the project includes an adapter for it.
- **Studies of SOC analysts** [4], [15], [16], [17]: used in Chapter 6 to build user personas through design thinking.
- **scikit-learn documentation and the original Isolation Forest, LOF and autoencoder papers** [18], [19], [20], [21]: reference for the machine-learning models used.

3. IDENTIFICATION OF THE PROBLEM / SCOPE OF THE PROJECT

3.1 Problem Identification

Attacks that use valid accounts, such as stolen credentials, account takeover and malicious insiders, produce log entries that look legitimate one at a time. Signature-based tools therefore miss them, and the damage is often discovered weeks later [2]. Behaviour-based detection can find them, but existing approaches have practical problems:

- **Too many false alarms.** Real behaviour is irregular, and analysts already report that most alerts are false or benign [4]. A detector with a high AUC can still have a precision below 1% [12].
- **Alerts that do not explain themselves.** Analysts distrust scores they cannot understand [17] and need context to decide quickly [16].
- **Users only.** Most research models user accounts and ignores devices, although some attacks (cryptomining, a compromised laptop used as a pivot) show up on machines.
- **Static or unprotected baselines.** A baseline learned once becomes stale as people change jobs and habits; a baseline that learns everything can be slowly poisoned by an attacker.
- **No prioritisation.** An analyst needs to know which alert to open first, not just which events are anomalous.

Problem statement: to develop a Python-based UEBA system that captures authentication, system-access, network and resource-usage logs, builds behavioural profiles for users and devices, identifies deviations from normal behaviour, and generates risk-based security alerts that are accurate, explained and prioritised.

3.2 Scope of the Project

In scope:

- Collection and normalisation of four log streams, including live capture of resource and connection data from the host machine.
- A simulated 48-person organisation (60 days, about 205,000 events) with labelled attacks for objective evaluation, and an adapter for the CERT data set.
- Daily behavioural profiles for 48 users and 60 devices, with personal and peer-group baselines.
- Three detection layers (statistical, unsupervised ML ensemble, MITRE-mapped rules), risk fusion, explained alerts and an analyst feedback loop.
- A Streamlit dashboard for security analysts and a full evaluation with robustness and ablation experiments.

Out of scope: real-time event streaming, automated response actions (for example disabling an account), deployment on a production network, and inspection of file or e-mail content (only metadata is analysed, for privacy).

4. PROPOSED IDEA / SOLUTION

The proposed solution is a layered UEBA pipeline that turns raw logs into a short list of explained, risk-ranked alerts. It works in five stages (Figure 6.1 shows the architecture):

1. **Capture.** Authentication, system-access, network and resource-usage logs are loaded, validated and merged into one event timeline.
2. **Behavioural features.** Each user's day is summarised into 44 numbers (for example failed logins, logins at unusual hours, MB uploaded to new domains) and each device's day into 19 numbers. Features such as "new host" and "unusual hour" are computed only from earlier days, as a live system would.
3. **Profiles.** Every entity gets a personal baseline (its own normal) and a peer baseline (its department's normal), using robust statistics. The personal baseline adapts over a rolling 45-day window, but days that look like attacks are quarantined.
4. **Detection.** Three independent layers score each day: robust z-score statistics, an unsupervised ML ensemble (Isolation Forest, Local Outlier Factor and an autoencoder) trained only on normal days, and 20 threat rules mapped to MITRE ATT&CK.
5. **Risk and response.** The layer scores are fused into a 0-100 daily risk, weighted by how critical the account or device is, and accumulated with decay over days to catch slow attacks. An alert is raised at a risk of 45 or more, with severity, plain-English reasons and ATT&CK techniques, and shown in a dashboard where analysts can mark it benign or confirmed.

Three ideas make the solution practical. First, **corroboration**: the threshold is set so that statistical deviation alone can never raise an alert; the ML ensemble or a rule must agree, which removes most false alarms caused by one unusual number. Second, **explainability**: every alert lists what was unusual and by how much compared with the entity's own history. Third, **learning with protection**: profiles keep adapting to legitimate change, while high-risk days, rule matches and analyst-confirmed threats are never learned.

5. OBJECTIVES

The objectives of the project are:

1. To capture and normalise authentication, system-access, network and resource-usage logs into a single, consistent event schema.
2. To build daily behavioural profiles for both users and devices, with personal and department peer-group baselines that adapt over time and resist poisoning.
3. To detect deviations from normal behaviour using complementary statistical, unsupervised machine-learning and rule-based methods, without requiring labelled attack data.
4. To combine all evidence into a single 0-100 risk score that accounts for asset criticality and slow, multi-day behaviour, and to generate alerts with plain-English reasons and MITRE ATT&CK mapping.
5. To provide an analyst dashboard for triage, investigation and feedback.
6. To evaluate the system objectively, aiming to detect every planted attack while keeping precision above 0.9 and false alerts to about one per day or fewer, and to check robustness across different simulated organisations.

6. PROCESS / METHODOLOGY

The project followed the design-thinking cycle (empathise, define, ideate, prototype, test) before and during implementation. This chapter moves from the users' needs (6.1) to product features (6.2), the specification (6.3), the detailed design (6.4) and the diagrams (6.5).

6.1 Identify the Needs of Customer and Users through Design Thinking Methods

Empathise

The primary users of a UEBA system are security operations centre (SOC) analysts; secondary stakeholders are the security manager who runs the SOC and the employees whose behaviour is monitored. It was not possible to interview working SOC analysts for this project, so the personas below are composites built from four published studies of SOC analysts, and are stated as such:

- A survey (n = 20) and interviews (n = 21) of SOC practitioners found that tools produce many false alarms needing manual validation, that most are benign triggers caused by legitimate behaviour, and that alarm validation causes burnout. Analysts asked for alarms that are Reliable, Explainable, Analytical, Contextual and Transferable (REACT) [4].
- Interviews with 17 professionals from 9 organisations showed that detection rules are judged mostly by how many alerts and false positives they generate, and that no interviewee saw replacing rules with AI-based anomaly detection as the answer [15].
- An experiment with 10 tier-1 analysts on 200 alerts found that analysts who followed a structured analysis process were 2.5 times more likely to reach an accurate assessment than those who did not [16].
- A study of six US Navy SOC analysts using ML-based tools found that they lacked a mental model of how scores were produced, which led to mistrust or misuse of the tools [17].

Table 6.1: User personas (composites built from published research)

Persona	Goals	Pain points	Needs from the system
Priya, tier-1 SOC analyst	Triage the queue quickly; never miss a real attack	Flood of false and benign alerts; scores with no reason; relies on experience rather than a structured process	Few, ranked alerts; a reason for every alert; context (usual vs today) in one place
Rahul, SOC manager / CISO	Balance analyst workload against detection coverage; report risk to management	Rules tuned only to reduce noise; sceptical of black-box AI; no overview of what is covered	Risk ranking, severity, MITRE ATT&CK coverage, measurable performance
Ananya, monitored employee	Do her job without being wrongly accused	Fear of intrusive monitoring; unusual but legitimate work (a new project) being flagged	Metadata-only monitoring, fair comparison with her own history and peers, quick clearing of false alerts

Table 6.2: Empathy map for the SOC analyst persona

Says	Thinks
"Most of these alarms are false." "Why did the tool flag this?"	"If I ignore alerts, I might miss the real one." "I cannot trust a score I don't understand."

Does	Feels
Opens each alert, searches logs manually, compares with what the user normally does, escalates or closes	Overloaded, desensitised to alarms, uncertain when closing an alert

Define

How might we help a SOC analyst spot account misuse and insider activity early, without adding to the flood of false alarms, and show clearly why each alert was raised?

Ideate

Three directions were considered. Signature rules alone would be precise and explainable but blind to anything without a rule. A single deep-learning model could find novel patterns but would be a black box, which the studies above show analysts distrust. The chosen direction was a hybrid: statistics and rules for explanation and precision, an unsupervised ML ensemble for unknown patterns, and a fused risk score that requires agreement between them.

Prototype and Test

A first working prototype detected every planted attack but raised 67 alerts in 30 days with a precision of only 0.31, mostly because legitimate monthly uploads were flagged. Testing against the users' needs led to three design changes: a baseline spread that respects each person's own occasional peaks, an adaptive baseline with quarantine, and a threshold that requires corroboration between layers. A later fix to the accumulated-risk calculation removed alerts that lingered after incidents. Together these raised precision to 0.964 (Chapter 10). Design thinking also added a feature: an analyst feedback button, so that a false alert can be closed and learned from.

6.2 Translate the Needs of Customer into Product

Table 6.3: User needs translated into product features

User need	Product feature	Where implemented
Fewer false alarms (Priya, Rahul)	Corroboration threshold of 45; P95-aware spread; adaptive baselines	risk.py, profiles.py, pipeline.py
Know why an alert was raised (Priya)	Plain-English reasons, per-layer score chart, MITRE technique IDs	detectors.py, dashboard
Context in one place (Priya)	Alert card with usual vs today values, risk timeline, login-hour chart, event timeline	dashboard (Alerts, Investigate)
Know what to open first (Priya, Rahul)	0-100 risk score, asset criticality, severity levels	risk.py
Catch slow insiders (Rahul)	Accumulated risk with a 3-day half-life	risk.py
Trust in automation (Rahul)	ML used as one corroborating layer alongside rules, never alone	pipeline.py
Close and learn from false alerts (Priya, Ananya)	"Mark benign" / "Confirm threat" buttons feeding back into baselines	feedback.py, dashboard
Privacy and fairness (Ananya)	Metadata-only daily counts; comparison with own history and own department	features.py, profiles.py
Coverage reporting (Rahul)	Overview page with alerts by ATT&CK technique; model performance page	dashboard

6.3 Specification of the Product

Functional requirements

Table 6.4: Functional requirements

ID	Requirement
FR1	Load authentication, access, network and resource logs from CSV or JSON-lines files and validate their columns.
FR2	Compute 44 daily features per user and 19 per device.
FR3	Learn personal and department peer baselines and update personal baselines over a rolling 45-day window.
FR4	Score every entity-day with the statistical layer, the ML ensemble and 20 threat rules.
FR5	Produce a 0-100 daily risk and a decaying accumulated risk per entity.
FR6	Raise alerts with severity, reasons and MITRE ATT&CK techniques when risk reaches the threshold.
FR7	Let an analyst mark an alert benign or confirmed and use this at the next analysis run.
FR8	Show overview, alerts, entity investigation, model performance, live host monitor and log explorer pages.
FR9	Measure precision, recall, F1-score, false positive rate, ROC-AUC and PR-AUC against ground truth.

Non-functional requirements and key parameters

- **Performance:** analyse about 205,000 events for 108 entities in under one minute on a laptop.
- **Explainability:** every alert must state its reasons; no black-box-only decisions.
- **Configurability:** all thresholds and weights kept in one configuration file.
- **Privacy:** only metadata (counts, hosts, domains, volumes) is processed, never file or message content.
- **Reliability:** behaviour verified by 11 automated tests.

Table 6.5: Key system parameters

Parameter	Value	Purpose
Baseline period	Days 6-30 (days 1-5 warm-up)	Learn normal behaviour
Adaptive window	Last 45 accepted days	Keep profiles current
Deviation threshold	$z \geq 3$	A feature counts as unusual
Layer weights	statistics 0.28, peer 0.10, ML 0.30	Fusion into daily risk
Alert threshold	Daily risk ≥ 45	Statistics alone cannot alert
Quarantine	Risk ≥ 60 or any rule hit	Protect baselines from poisoning
Criticality	Admin/server 1.3, Finance/Management 1.2, HR 1.1, others 1.0	Reflect impact of compromise
Risk half-life	3 days	Accumulate slow behaviour
Impossible travel	> 500 km at > 900 km/h	Faster than a commercial flight

6.4 Designing the Solution for the Customer Needs

6.4.1 Log capture

Four log streams are used. Authentication logs record the time, user, source device and IP, destination host, method (interactive, VPN, SSH, RDP, Kerberos), result and location. Access logs record the resource, action (read, write, delete, execute, privilege escalation) and sensitivity label. Network logs record the destination domain, port and bytes in and out. Resource logs record hourly CPU, memory, disk and process counts. For evaluation a simulated organisation produces these logs (Chapter 9); the *collector* module captures real resource and connection data from the host using psutil, and an adapter converts the CERT data set [14] into the same format.

6.4.2 Behavioural features

Table 6.6: Behavioural features per user-day

Group	Examples	Detects
Authentication (14)	failed logins, failures in 10 min, success after failures, new hosts, logins at unusual hours, new countries, impossible travel, unregistered devices	brute force, stolen credentials, lateral movement
System access (12)	sensitive and restricted accesses, privilege escalations, new resources, account creation, log clearing, USB events	data theft, privilege abuse, covering tracks
Network (12)	MB uploaded, largest upload, new domains, MB to new external domains, cloud-storage uploads, mining-pool connections, job-site visits	exfiltration, cryptomining, insider risk
Resource (6)	average and peak CPU, hours above 85% CPU, memory, disk I/O, active hours	cryptomining, malware

A login hour is "unusual" when the user logged in at that hour on fewer than 2% of earlier days. Impossible travel uses the great-circle (haversine) distance between consecutive login locations divided by the time between them. Device features mirror these from the machine's side (logins into the device, new users on it, uploads, CPU).

6.4.3 Profiles

For each entity and feature the baseline stores a centre and a spread. Robust statistics are used because behaviour data is spiky:

$$centre = median(history) \qquad spread = max(1.4826 \times MAD, (P95 - median) / 1.645, floor)$$

$$z = (today - centre) / spread, \ \ clipped\ to\ [-10, 10]$$

The 95th-percentile term stops a person's normal occasional peaks (for example a monthly 60 MB presentation upload) from being flagged. The floor prevents division by zero for features that are always zero. A peer baseline is computed the same way over the department. During detection the personal baseline is recomputed each day from the last 45 accepted days; a day is not accepted if its risk is 60 or more, if it matched any threat rule, or if an analyst confirmed it as a threat. Days an analyst marks benign are always accepted.

6.4.4 Detection layers

- **Statistical layer:** the three largest positive deviations above z = 3 are combined into a score between 0 and 1, for both the personal and the peer baseline.
- **ML ensemble:** Isolation Forest [19] (300 trees), Local Outlier Factor [20] (25 neighbours, novelty mode) and an autoencoder [21] (44-16-6-16-44 neurons), implemented with scikit-learn [18], are trained on the z-score vectors of normal days only. Each score is calibrated against its training distribution and the three are averaged.
- **Threat rules:** 20 rules, for example brute force (10 or more failures in 10 minutes followed by success, T1110), impossible travel (T1078), privilege escalation by a non-admin (T1068), large upload to a new external domain (T1048), cloud-storage upload (T1567.002), mining-pool traffic (T1496) and log clearing (T1070). Each rule has a weight between 0.10 and 0.80.

6.4.5 Risk scoring and alerts

$$A = 1 - (1 - 0.28 \cdot stat)(1 - 0.10 \cdot peer)(1 - 0.30 \cdot ml)(1 - rules)$$

$$daily\ risk = 100 \times min(1, A \times criticality)$$

$$entity\ risk(t) = min(100, entity\ risk(t-1) \times 0.5\text{^}(days/3) + max(0, daily\ risk - 15))$$

The fusion behaves like a probabilistic OR: one strong signal raises risk, agreeing signals raise it further, and it never exceeds 100. The statistical layers alone can reach at most 43.8 (both at their maximum of 0.953, with criticality 1.3), so the alert threshold of 45 always requires the ML ensemble or a rule to agree. Alerts are critical at 80 or more, high at 60-79 and medium at 45-59. If accumulated risk reaches 60 without a recent alert, a "sustained risk" alert is raised to catch slow insiders.

6.4.6 Dashboard and feedback

The Streamlit dashboard has six pages: an overview of alerts and risk, an alert queue with an explained detail view and feedback buttons, an entity investigation page (profile, risk timeline, biggest deviations, login hours and event timeline), a model performance page with a threshold slider, a live host monitor and a log explorer. All results are stored in a single SQLite database.

6.5 Process / Flow / Design Diagram

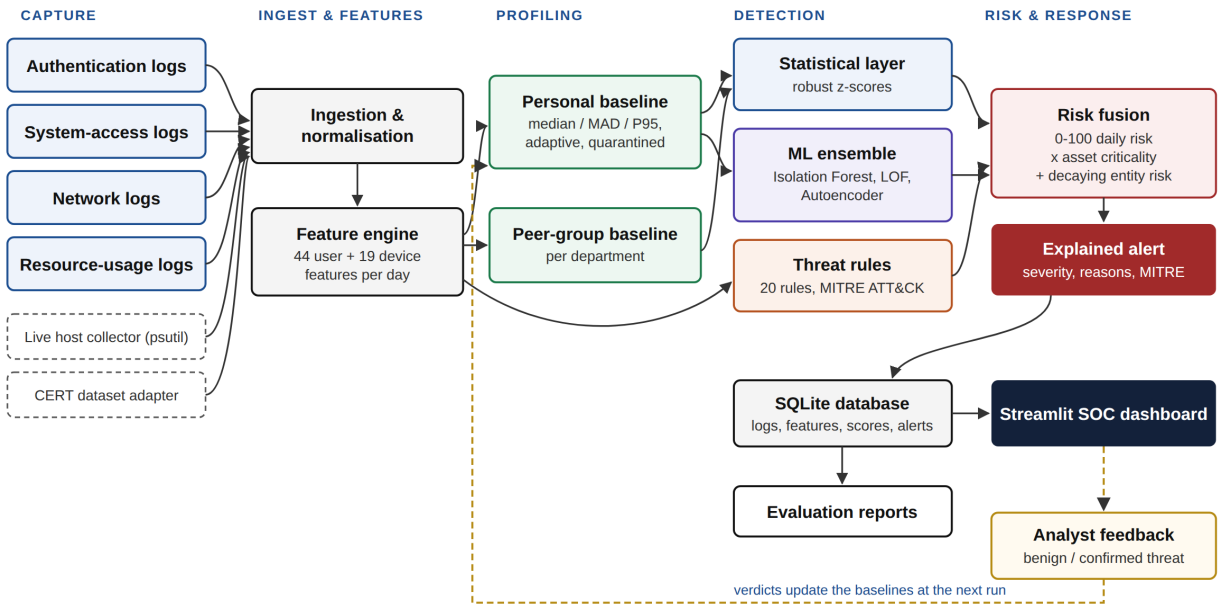


Figure 6.1: System architecture

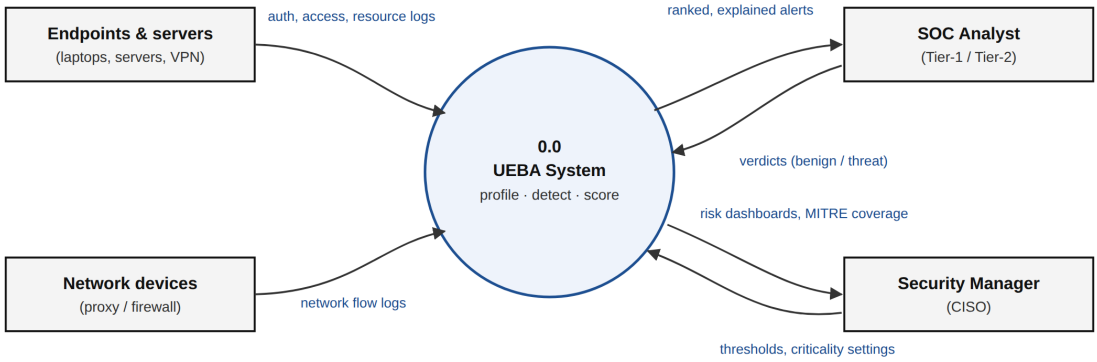


Figure 6.2: Data flow diagram, level 0 (context)

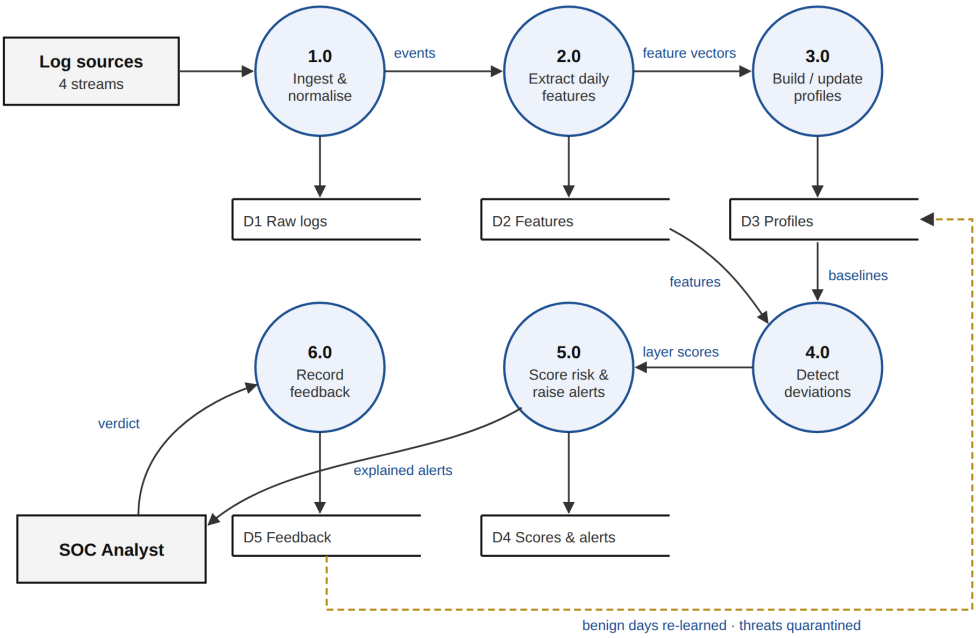


Figure 6.3: Data flow diagram, level 1

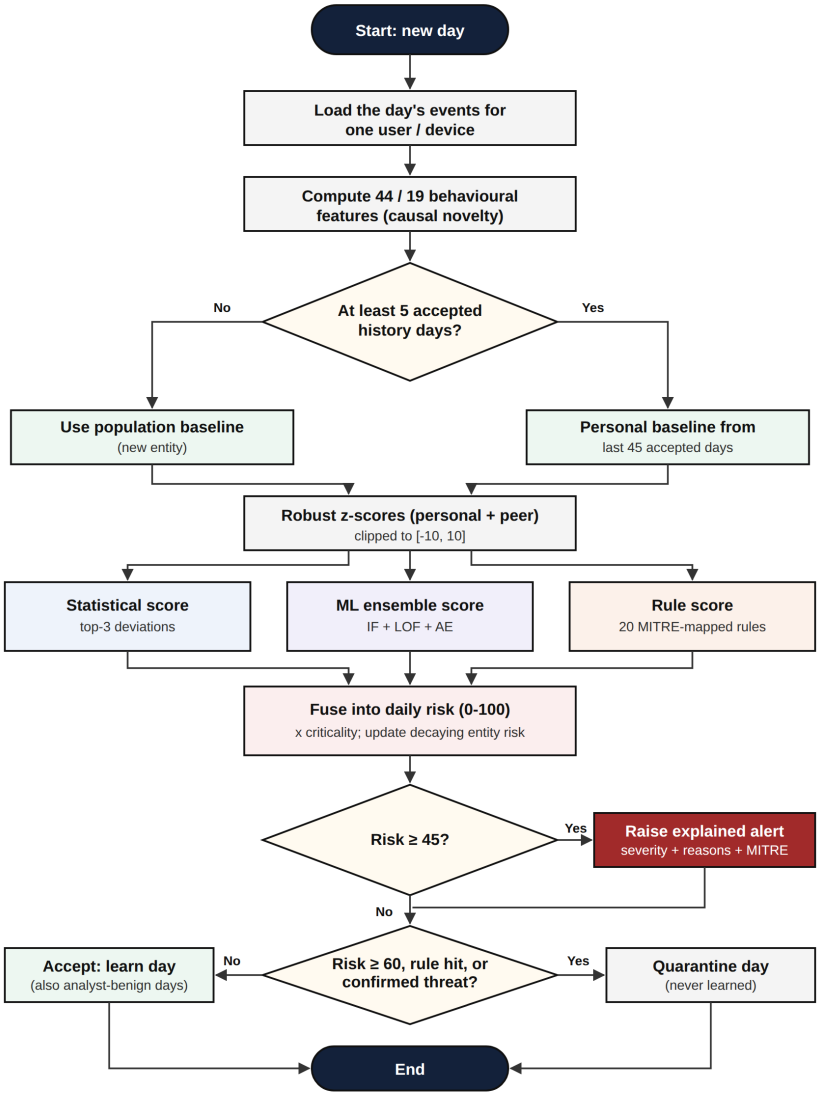


Figure 6.4: Flowchart of daily scoring for one entity

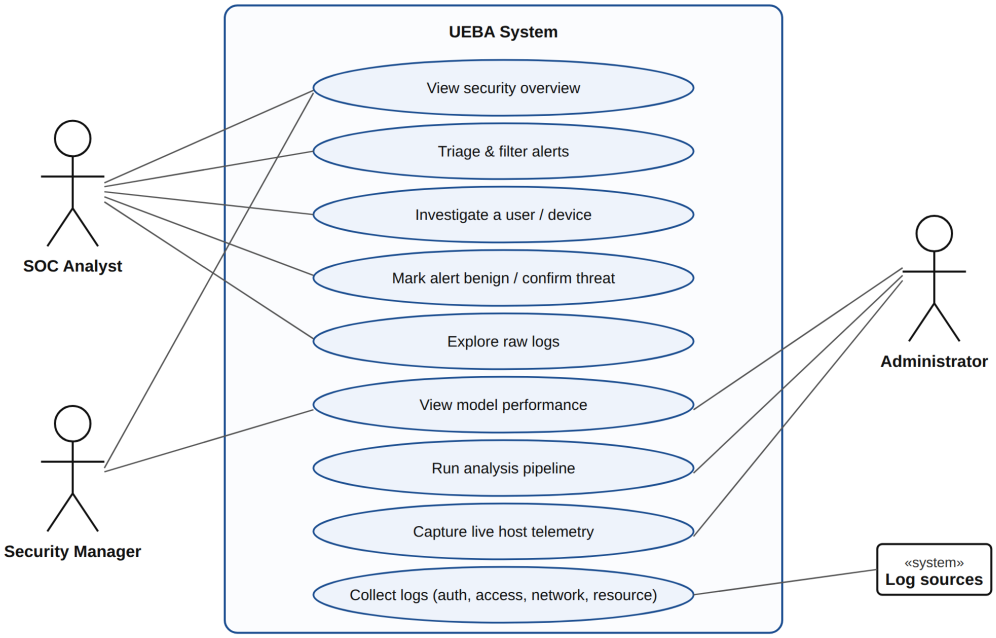


Figure 6.5: Use case diagram

7. REQUIREMENTS

7.1 Hardware Requirements

Table 7.1: Hardware requirements

Component	Minimum	Used for development
Processor	Dual-core, 64-bit	Apple M-series / x86-64 laptop
Memory (RAM)	4 GB	8 GB or more
Storage	1 GB free (15 GB if the CERT data set is used)	SSD
GPU	Not required	Not used

7.2 Software Requirements

Table 7.2: Software requirements

Software	Version	Purpose
Operating system	macOS, Linux or Windows	Development and execution
Python	3.10 or later	Programming language
pandas, NumPy	pandas 2.0+, NumPy 1.24+	Log processing and features
scikit-learn	1.3+	Isolation Forest, LOF, autoencoder (MLP)
Streamlit, Plotly	Streamlit 1.50+, Plotly 5.18+	Dashboard and interactive charts
Matplotlib	3.7+	Evaluation figures
SQLite	Bundled with Python	Storage of logs, scores and alerts
psutil	5.9+	Live host capture
pytest	7.4+	Automated tests
Web browser	Any modern browser	Viewing the dashboard

7.3 Datasets

- **Simulated enterprise logs (primary):** generated by the project for a 48-person organisation over 60 days: 9,347 authentication, 75,295 access, 85,432 network and 35,409 resource events (205,483 in total), with 11 labelled attacks (28 attack entity-days) and 4 unlabelled benign anomalies. Real organisational logs are confidential and never labelled, so simulation with planted attacks is what makes objective measurement possible.
- **CMU CERT Insider Threat Test Dataset r4.2 (optional)** [14]: the standard public benchmark (1,000 simulated employees, 70 malicious insiders). An adapter is implemented and tested on a sample in the same format; a full run is future work.
- **Live host data:** CPU, memory, process and connection samples captured from the machine running the dashboard.

7.4 Technologies and Tools

Table 7.3: Technologies and tools

Category	Technology / tool
Language	Python 3
Data processing	pandas, NumPy
Machine learning	scikit-learn (IsolationForest, LocalOutlierFactor, MLPRegressor autoencoder, StandardScaler)
Storage	SQLite
Dashboard	Streamlit, Plotly
Visualisation	Matplotlib
System telemetry	psutil
Testing	pytest
Threat framework	MITRE ATT&CK
Editor and tools	Visual Studio Code, Git, terminal

8. NOVELTY OF THE PROPOSED IDEA / PROJECT

The individual techniques used in the project are established. What is new is how they are combined to address the practical problems identified in Chapter 2. Table 8.1 compares the project with the reviewed work.

Table 8.1: Feature comparison with reviewed work

Capability	Reviewed work	This project
Users and devices profiled together	Users only in [9], [10], [11]; hosts only in [12]	Yes: 44 user and 19 device features
Personal and peer baselines	Relationships via graphs in [9]	Yes, with robust statistics
Adaptive baseline	Retraining in [3]	Yes, rolling 45 days
Protection against baseline poisoning	Not addressed	Yes: quarantine of risky days, rule hits and confirmed threats
No attack labels needed	Labels used in [9], [10]	Yes: fully unsupervised plus rules
Statistics, ML and rules corroborated	Single model in most work	Yes: threshold requires agreement
Explained alerts	Feature attribution in [7]; attention in [13]	Plain-English reasons, layer scores, MITRE IDs
Prioritised 0-100 risk with criticality and decay	Not provided	Yes
Analyst feedback updates profiles	Labels per cluster in [13]	Yes: benign / threat verdicts

The main points of novelty are:

1. **Poisoning-resistant adaptive baselines.** Profiles keep learning legitimate change, but days with high risk, rule matches or analyst-confirmed threats are never learned, while analyst-cleared days are.
2. **Corroboration by design.** The alert threshold is derived from the score formula so that a single statistical deviation can never alert on its own. This is the main reason precision rose from 0.31 in the first prototype to 0.96.
3. **Spread that respects personal peaks.** Using the 95th percentile alongside the MAD stops a person's normal occasional spikes from being treated as anomalies.
4. **One explained risk score for users and devices.** Statistical, ML and rule evidence, asset criticality and slow multi-day behaviour are combined into a single number, with the reasons attached in plain English.
5. **Designed from analyst needs.** Features were derived from published SOC studies through design thinking (Table 6.3), not added for their own sake.

9. EXPERIMENTATION

9.1 Experimental Setup

The experiments use a simulated organisation of 48 employees in six departments (Engineering 14, Sales 10, Finance 8, HR 6, IT-Admin 5, Management 5), each with a laptop, plus 12 servers, giving 108 monitored entities. Every person has individual habits: working hours, office or remote location, servers, files and websites used, upload volumes and CPU load. Weekends, leave days, mistyped passwords and occasional legitimate large uploads are included. The logs cover 60 days: days 1-5 are warm-up, days 6-30 form the attack-free baseline, and days 31-60 form the detection window.

Table 9.1: Attacks planted in the detection window

ID	Scenario	What appears in the logs	ATT&CK
INC-01	Brute force + takeover	45 failed VPN logins from a Tor exit node at 03:10, then success, file reads, 400 MB upload	T1110
INC-02	Impossible travel	Login in Bengaluru, then from Amsterdam 70 minutes later	T1078
INC-03	Off-hours access	Login at 02:20, 60 confidential files including payroll read, upload to WeTransfer	T1078
INC-04	Data exfiltration	90 files and production API keys read, 3.3 GB uploaded to mega.nz	T1567
INC-05	Slow insider	Five days of growing Dropbox uploads (120 MB to 1.3 GB) and job-site visits	T1567.002
INC-06	Privilege escalation	Sales user runs administrator commands on the domain controller	T1068
INC-07	Lateral movement	SSH/RDP into 12 never-seen hosts within 50 minutes	T1021
INC-08	Cryptomining	Laptop CPU at about 96% for three days; traffic to a mining pool on port 3333	T1496
INC-09	Password spray	30 failed logins on an administrator account, no success	T1110
INC-10	Travel + exfiltration	Login from Lagos 70 minutes after Bengaluru, then 1.2 GB upload	T1078, T1567
INC-11	Admin account abuse	Login from Frankfurt at 03:05, four accounts created, security log cleared	T1136, T1070

These produce 28 labelled attack entity-days (17 user-days and 11 device-days, because an attack run from a person's own laptop also labels that laptop). Four harmless but unusual events are also planted and **not** labelled, to test whether the system can tell "unusual" from "dangerous": an engineer working until 1 a.m., a salesperson logging in from Singapore for three days, an IT administrator sending 20 GB to the internal backup server, and an HR employee opening 11 files from another department for a new project.

9.2 Evaluation Protocol and Metrics

Each entity-day in the detection window (2,363 days with activity) is a test case: positive if it is a labelled attack day, and predicted positive if the system raised an alert for it. No labels are used for training; they are used only for scoring. The metrics are precision TP/(TP+FP), recall TP/(TP+FN), F1-score, false positive rate FP/(FP+TN), ROC-AUC and PR-AUC of each layer's score, and incident-level detection

(whether and when each attack was alerted). PR-AUC matters because attacks are only about 1% of entity-days, so accuracy would be misleading.

9.3 Experiments Conducted

1. **Main run:** the full system on the default organisation (random seed 42).
2. **Robustness:** the whole organisation regenerated with five random seeds (42, 7, 123, 2024, 99), each giving different people, habits, victims and timing.
3. **Ablation:** layers switched off in turn (no rules, no ML, each layer alone) and re-measured on the five seeds.
4. **Threshold sweep:** precision and recall measured for alert thresholds from 20 to 80.
5. **Functional tests:** 11 automated tests covering distance calculation, unusual-hour logic, rules, risk-score bounds and corroboration, the full pipeline, the analyst feedback loop and the CERT adapter on a sample in the CERT file format.

All experiments were run in Python 3.11; one full analysis of about 205,000 events takes roughly 20-30 seconds on a laptop.

10. RESULTS AND DISCUSSION

10.1 Main Results

Table 10.1: Detection results on the main run (detection window)

Metric	Value
Entity-days scored / attack entity-days	2,363 / 28
Alerts raised	28 (26 critical, 1 high, 1 medium), about 0.9 per day
TP / FP / FN / TN	27 / 1 / 1 / 2,334
Precision / Recall / F1-score	0.964 / 0.964 / 0.964
False positive rate	0.04%
Incidents detected	11 of 11, all on the day they started
Fused risk ROC-AUC / PR-AUC	1.000 / 0.999

Table 10.2: Ranking quality of each layer (main run)

Detector	ROC-AUC	PR-AUC
Fused risk score	1.000	0.999
Personal baseline (z-score)	0.997	0.926
Peer-group baseline	0.982	0.956
ML ensemble	0.998	0.855
Isolation Forest	0.869	0.455
Local Outlier Factor	0.909	0.734
Autoencoder	0.996	0.633
Threat rules	0.982	0.965

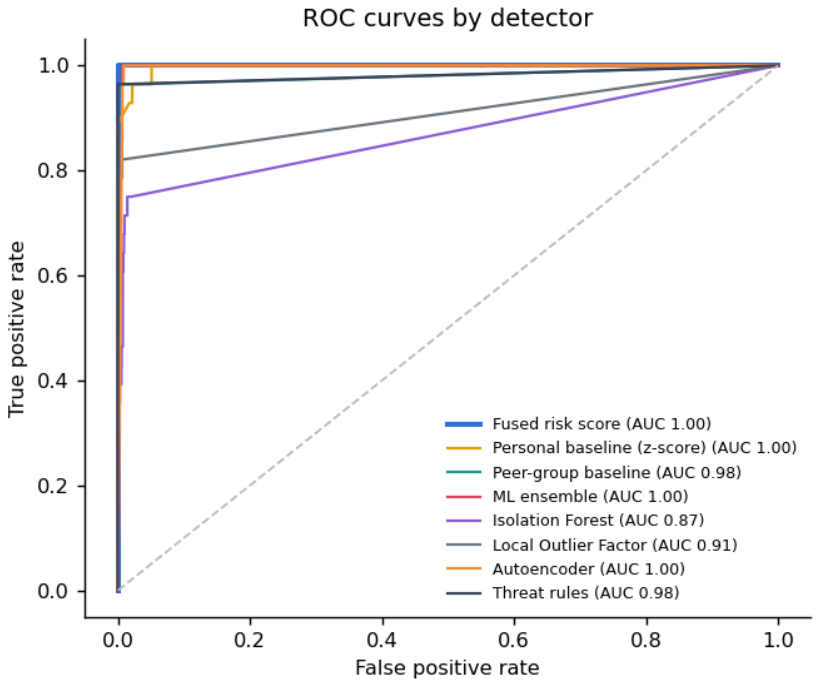


Figure 10.1: ROC curves of each detection layer

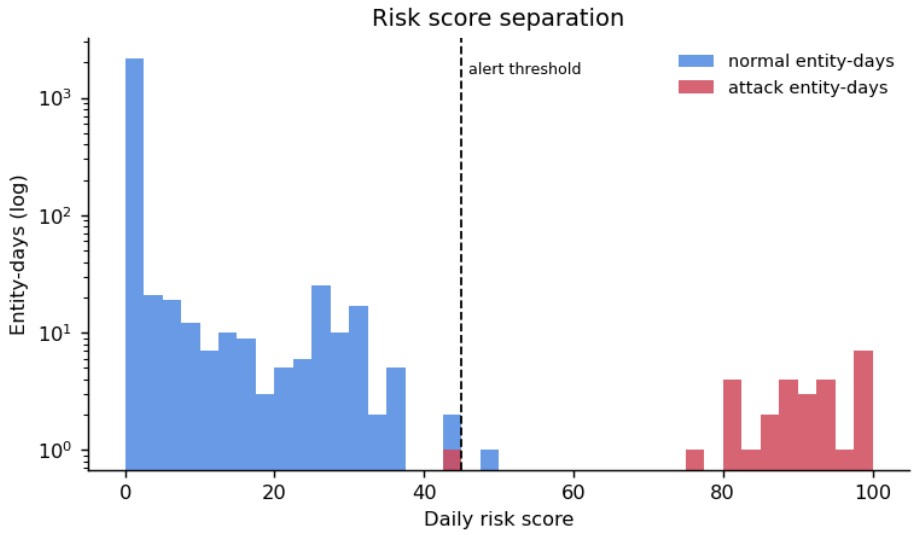


Figure 10.2: Daily risk of normal and attack entity-days (dashed line: alert threshold 45)

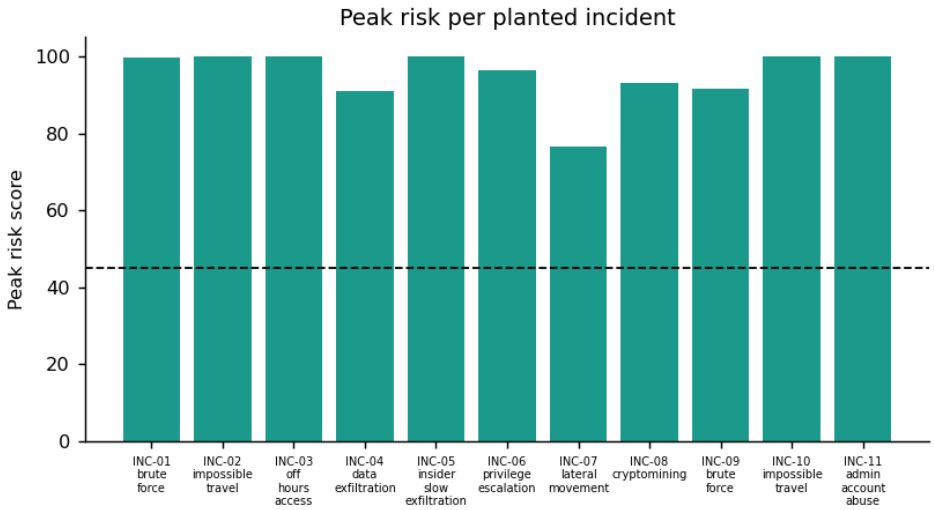


Figure 10.3: Peak risk of each planted incident

Every incident reached a peak risk above the threshold; ten were critical and the lateral-movement attack was high (76.6). The single false positive was the planted "new project" case: an HR employee opened 11 files from another department (risk 49.4). It is genuinely unusual behaviour, and the alert's reasons make it quick for an analyst to close with the feedback button. The single false negative was the laptop used as the pivot for lateral movement, which scored 43.6, just below the threshold; the user account carrying out the lateral movement was alerted on the same day, so the incident itself was caught. The other three harmless anomalies (late-night work, the business trip and the large internal backup) did not raise alerts.

10.2 Robustness Across Organisations

Table 10.3: Results for five differently generated organisations

Seed	Incidents	Precision	Recall	F1-score	FPR	Alerts/day
42	11/11	0.964	0.964	0.964	0.04%	0.93
7	11/11	0.931	0.964	0.947	0.09%	0.97
123	11/11	0.964	0.964	0.964	0.04%	0.93
2024	11/11	0.931	0.964	0.947	0.08%	0.97
99	11/11	0.900	0.964	0.931	0.13%	1.00
Mean	55/55	0.938	0.964	0.951	0.08%	0.96

All 55 planted incidents were detected across the five organisations, with a mean F1-score of 0.951 and about one alert per day for 108 entities. The results are therefore not specific to one random organisation.

10.3 Ablation Study

Table 10.4: Contribution of each layer (mean ± standard deviation over five seeds)

Variant	PR-AUC	Precision	Recall	F1-score
Full system	0.996 ± 0.002	0.938 ± 0.027	0.964 ± 0.000	0.951 ± 0.014
Without threat rules	0.951 ± 0.007	0.774 ± 0.124	0.971 ± 0.016	0.857 ± 0.086
Without ML ensemble	0.984 ± 0.001	0.979 ± 0.020	0.964 ± 0.000	0.971 ± 0.010
ML ensemble only	0.856 ± 0.074	0.763 ± 0.080	0.821 ± 0.036	0.789 ± 0.048
Personal baseline only	0.457 ± 0.047	0.295 ± 0.027	0.971 ± 0.016	0.451 ± 0.031
Threat rules only	0.965 ± 0.000	1.000 ± 0.000	0.964 ± 0.000	0.982 ± 0.000

The ablation needs careful reading. The rules alone look almost perfect only because they were written with these attack types in mind; against an attack with no matching rule they score nothing. With all rules switched off, the behavioural layers still reached an F1-score of 0.857 with no prior knowledge of the attacks, which is the real value of the UEBA approach. Statistics alone flag almost everything unusual (recall 0.971) but with poor precision (0.295), which justifies the corroboration threshold. On this data, removing the ML ensemble slightly raises F1 because the rules already cover every planted attack; the ensemble improves ranking (PR-AUC) and matters for attacks that no rule describes.

10.4 Choice of Threshold

Table 10.5: Effect of the alert threshold (main run, daily-risk alerts)

Threshold	Precision	Recall	F1-score	Alerts
30	0.509	1.000	0.675	55
35	0.778	1.000	0.875	36
40	0.903	1.000	0.949	31
45 (chosen)	0.964	0.964	0.964	28
50	1.000	0.964	0.982	27
80	1.000	0.929	0.963	26

A threshold of 50 would have removed the one false positive on this data. The threshold was nevertheless kept at 45 because it follows from the design rule that statistics alone must not alert, rather than being

tuned to the test results, which would overfit the evaluation. The dashboard lets an analyst explore this trade-off live.

10.5 Dashboard

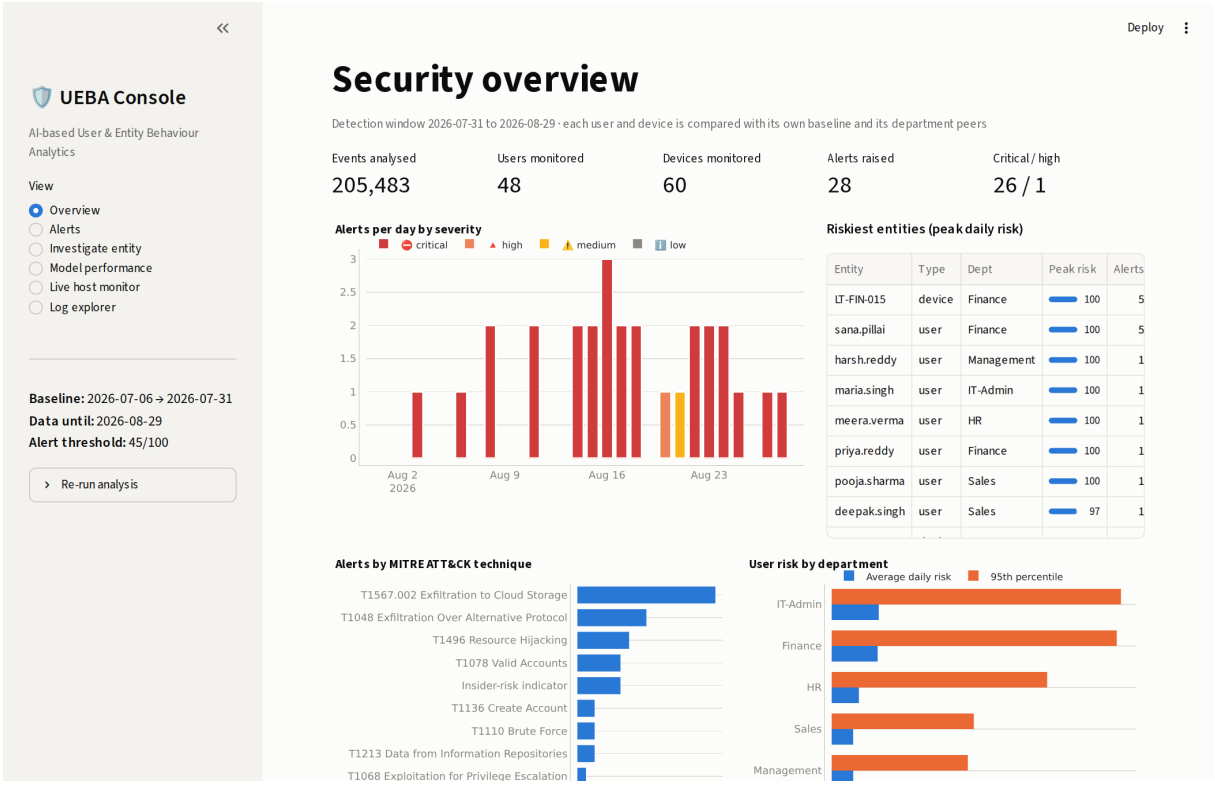


Figure 10.4: Dashboard overview page

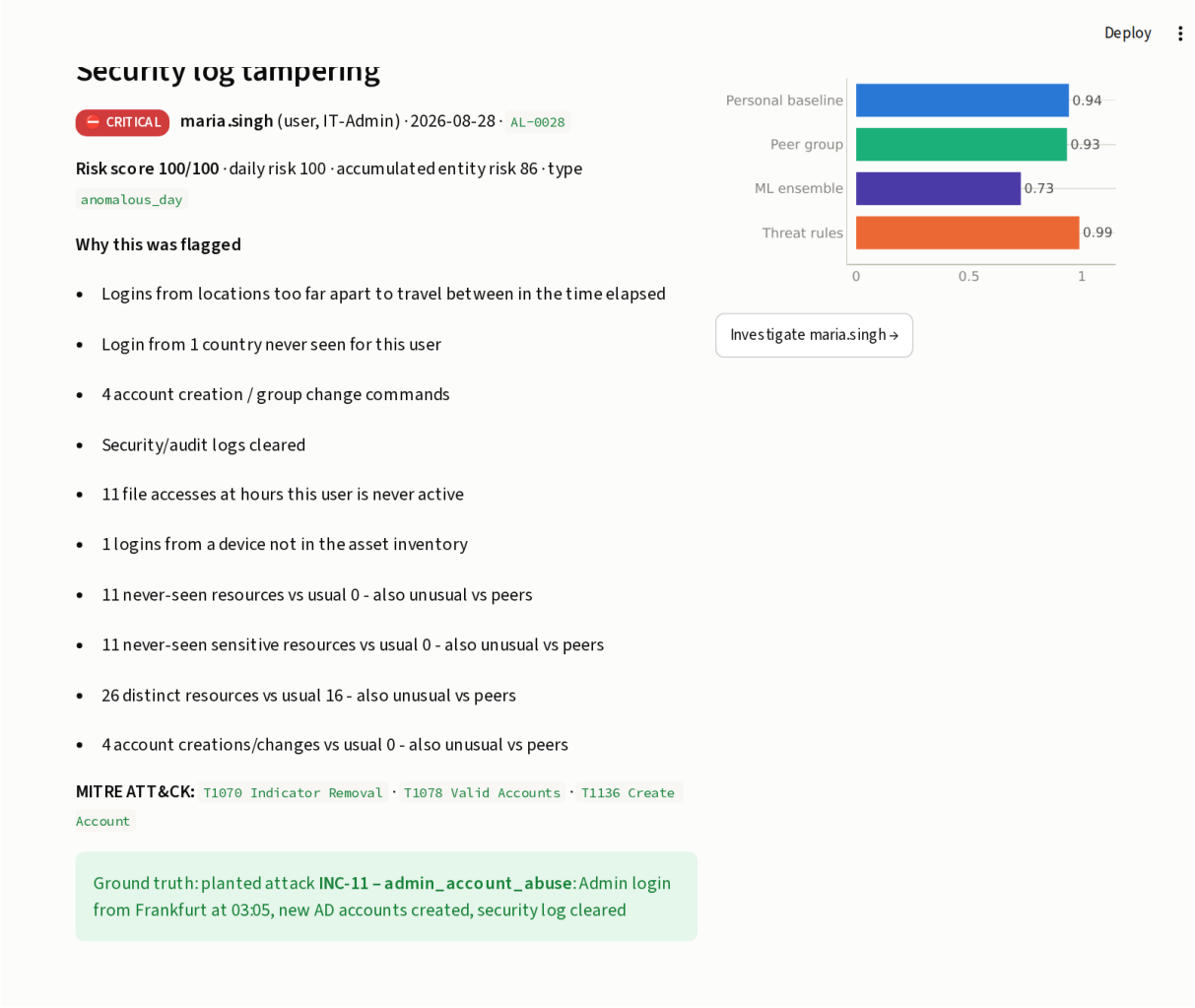


Figure 10.5: Alert detail with reasons, ATT&CK techniques and layer scores

10.6 Discussion

The results show that the system meets its objectives on the simulated organisation: all attacks were detected on the first day, with about one alert per day and clear reasons for each. They should not be compared directly with the CERT results in Chapter 2, because the data sets differ. The simulated attacks were also designed by the same person who built the detector, and Landauer et al. note that synthetic data tends to give lower false alarm rates than real behaviour [3]; on real logs, precision would be expected to fall. Euler's precision of about 0.006 on real data [12] shows how hard this is, and why the corroboration and feedback features matter. The main limitations are the simulated evaluation data, daily rather than real-time scoring, and rules that cover only known techniques.

11. CONCLUSION AND FUTURE DIRECTIONS

11.1 Conclusion

This project developed a Python-based User and Entity Behaviour Analytics system that meets the problem statement. It captures authentication, system-access, network and resource-usage logs; builds daily behavioural profiles for 48 users and 60 devices with personal and peer baselines that adapt over time but resist poisoning; detects deviations with three complementary layers (robust statistics, an unsupervised Isolation Forest, LOF and autoencoder ensemble, and 20 MITRE ATT&CK rules); and turns the evidence into a single 0-100 risk score with explained, prioritised alerts in an analyst dashboard with a feedback loop.

On 60 days of simulated logs with 11 planted attacks, the system detected every attack on the day it began, with precision, recall and F1-score of 0.964 and one false alert in 30 days. Across five differently generated organisations the mean F1-score was 0.951, and without any rules the behavioural layers alone reached 0.857. The design-thinking process, grounded in published studies of SOC analysts, shaped the most important features: corroboration between layers, plain-English reasons and analyst feedback. These raised precision from 0.31 in the first prototype to 0.96.

11.2 Future Directions

- **Evaluation on real and benchmark data:** run the existing adapter on the full CERT r4.2 data set and evaluate on the real CLUE-LDS logs [3] to obtain results comparable with published work.
- **Real-time detection:** stream events (for example with Apache Kafka) and score them as they arrive instead of once a day.
- **Sequence and graph models:** add an LSTM over event order [8] and user-host graph features for lateral movement [12] as further corroborating layers.
- **Automated response:** integrate with a SOAR platform to disable an account or isolate a host when a critical alert is confirmed.
- **Real deployment:** ingest Windows Security events, Sysmon, VPN and proxy logs, with role-based access to the dashboard and a monitoring policy compliant with data-protection law.

REFERENCES

[1] Verizon Business, "2025 Data Breach Investigations Report," Verizon, 2025. [Online]. Available: https://www.verizon.com/business/resources/reports/2025-dbir-data-breach-investigations-report.pdf

[2] Ponemon Institute and DTEX Systems, "2025 Cost of Insider Risks Global Report," Feb. 2025. [Online]. Available: https://www.globenewswire.com/news-release/2025/02/25/3032261/0/en/Ponemon-Cybersecurity-Report-Insider-Risk-Management-Enabling-Early-Breach-Detection-and-Mitigation.html

[3] M. Landauer, F. Skopik, G. Höld, and M. Wurzenberger, "A user and entity behavior analytics log data set for anomaly detection in cloud computing," in Proc. IEEE Int. Conf. Big Data (Big Data), Osaka, Japan, Dec. 2022, pp. 4285-4294, doi: 10.1109/BigData55660.2022.10020672.

[4] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in Proc. 31st USENIX Security Symp., Boston, MA, USA, Aug. 2022, pp. 2783-2800.

[5] The MITRE Corporation, "MITRE ATT&CK," 2026. [Online]. Available: https://attack.mitre.org/

[6] J. Cui, G. Zhang, Z. Chen, and N. Yu, "Multi-homed abnormal behavior detection algorithm based on fuzzy particle swarm cluster in user and entity behavior analytics," Sci. Rep., vol. 12, Art. no. 22349, Dec. 2022, doi: 10.1038/s41598-022-26142-w.

[7] J. Fuentes, I. Ortega-Fernández, N. M. Villanueva, and M. Sestelo, "Cybersecurity threat detection based on a UEBA framework using deep autoencoders," AIMS Math., vol. 10, no. 10, pp. 23496-23517, 2025, doi: 10.3934/math.20251043.

[8] M. Villarreal-Vasquez, G. Modelo-Howard, S. Dube, and B. Bhargava, "Hunting for insider threats using LSTM-based anomaly detection," IEEE Trans. Dependable Secure Comput., vol. 20, no. 1, pp. 451-462, 2023, doi: 10.1109/TDSC.2021.3135639.

[9] X. Li, X. Li, J. Jia, L. Li, J. Yuan, Y. Gao, and S. Yu, "A high accuracy and adaptive anomaly detection model with dual-domain graph convolutional network for insider threat detection," IEEE Trans. Inf. Forensics Security, vol. 18, pp. 1638-1652, 2023, doi: 10.1109/TIFS.2023.3245413.

[10] X. Cai, Y. Wang, S. Xu, H. Li, Y. Zhang, Z. Liu, and X. Yuan, "LAN: Learning adaptive neighbors for real-time insider threat detection," IEEE Trans. Inf. Forensics Security, vol. 19, pp. 10157-10172, 2024, doi: 10.1109/TIFS.2024.3488527.

[11] X. Zhu, J. Dong, J. Qi, Z. Zhou, Z. Dong, Y. Sun, and M. Wang, "AUTH: An adversarial autoencoder based unsupervised insider threat detection scheme for multisource logs," IEEE

Trans. Ind. Informat., vol. 20, no. 9, pp. 10954-10965, Sep. 2024, doi: 10.1109/TII.2024.3393491.

[12] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," ACM Trans. Privacy Security, vol. 26, no. 3, pp. 1-36, 2023, doi: 10.1145/3588771.

[13] T. van Ede et al., "DEEPCASE: Semi-supervised contextual analysis of security events," in Proc. IEEE Symp. Security Privacy (SP), San Francisco, CA, USA, May 2022, pp. 522-539, doi: 10.1109/SP46214.2022.9833671.

[14] B. Lindauer, "Insider Threat Test Dataset," Carnegie Mellon Univ., Software Engineering Institute, 2020, doi: 10.1184/R1/12841247.v1.

[15] M. Vermeer, N. Kadenko, M. van Eeten, C. Gañán, and S. Parkin, "Alert alchemy: SOC workflows and decisions in the management of NIDS rules," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Copenhagen, Denmark, Nov. 2023, pp. 2770-2784, doi: 10.1145/3576915.3616581.

[16] L. Kersten, T. Mulders, E. Zambon, C. Snijders, and L. Allodi, "'Give me structure': Synthesis and evaluation of a (network) threat analysis process supporting tier 1 investigations in a security operation center," in Proc. 19th Symp. Usable Privacy Security (SOUPS), Anaheim, CA, USA, Aug. 2023, pp. 97-111.

[17] S. Oesch et al., "An assessment of the usability of machine learning based tools for the security operations center," in Proc. IEEE Int. Conf. iThings/GreenCom/CPSCom/SmartData/Cybermatics, Rhodes, Greece, Nov. 2020, pp. 634-641, doi: 10.1109/iThings-GreenCom-CPSCom-SmartData-Cybermatics50389.2020.00111.

[18] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[19] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413-422.

[20] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in Proc. ACM SIGMOD Int. Conf. Management of Data, Dallas, TX, USA, 2000, pp. 93-104.

[21] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," Science, vol. 313, no. 5786, pp. 504-507, 2006.

SOURCE CODE

The complete source code (about 3,000 lines of Python in 13 modules, a Streamlit dashboard and 11 automated tests) is in the project folder. The key parts of the detection logic are reproduced below.

Risk fusion and accumulated risk (ueba/risk.py)

```
def daily_risk(df: pd.DataFrame, cfg: Config = CONFIG) -> np.ndarray:
    a = 1 - ((1 - cfg.w_stat * df["stat"]) * (1 - cfg.w_peer * df["peer"]) *
             (1 - cfg.w_ml * df["ml"]) * (1 - df["rule"]))
    crit = df["department"].map(cfg.criticality).fillna(1.0)
    crit = np.where(df["is_admin"].astype(bool), np.maximum(crit, cfg.criticality.get("IT-Admin",
1.3)), crit)
    return np.round(100 * np.clip(a * crit, 0, 1), 1)


def entity_risk(df: pd.DataFrame, cfg: Config = CONFIG) -> np.ndarray:
    """Decayed accumulation of daily risk above the noise floor (per entity, chronological)."""
    out = np.zeros(len(df))
    decay_per_day = 0.5 ** (1 / cfg.risk_half_life_days)
    order = df.sort_values(["entity", "date"]).index
    prev_ent, prev_date, r = None, None, 0.0
    dates = pd.to_datetime(df["date"])
    for idx in order:
        ent, d = df.at[idx, "entity"], dates.at[idx]
        if ent != prev_ent:
            r = 0.0
        else:
            r *= decay_per_day ** max(1, (d - prev_date).days)
        r = min(100.0, r + max(0.0, df.at[idx, "daily_risk"] - NOISE_FLOOR))
        out[df.index.get_loc(idx)] = r
        prev_ent, prev_date = ent, d
    return np.round(out, 1)
```

Robust baseline spread (ueba/pipeline.py)

```
def _robust(hist: np.ndarray, floor: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Median and robust scale of an entity's history (same formula as profiles._scales)."""
    med = np.median(hist, axis=0)
    mad = np.median(np.abs(hist - med), axis=0) * 1.4826
    tail = (np.quantile(hist, 0.95, axis=0) - med) / 1.645
    return med, np.maximum(np.maximum(mad, tail), floor)
```

Adaptive baseline with quarantine and analyst feedback (excerpt of score_entities) (ueba/pipeline.py)

```
    accepted = train_mask.copy()
    rows_by_entity = {e: np.array(ix) for e, ix in F.groupby("entity").indices.items()}
    dates = F["date"].to_numpy()
    for day in sorted(F.loc[~train_mask, "date"].unique()):
        idx = np.flatnonzero((dates == day))
        for i in idx:
            ix = rows_by_entity[F.at[i, "entity"]]
            hist = ix[(dates[ix] < day) & accepted[ix]][-cfg.history_days:]
            if len(hist) >= 5:
                med[i], scale[i] = _robust(X[hist], floor)
        score_rows(idx)
        # quarantine high-risk days and anything matching a threat signature; medium,
        # purely behavioural deviations are absorbed so the profile adapts to new habits
        accepted[idx] = ((S.loc[idx, "daily_risk"].to_numpy() < cfg.quarantine_risk)
                         & (S.loc[idx, "rule"].to_numpy() < 0.2))
        for i in idx:
            key = (F.at[i, "entity"], day)
            if benign and key in benign:
                accepted[i] = True
            elif threat and key in threat:
                accepted[i] = False
```

Statistical score and ML ensemble (ueba/detectors.py)

```
def deviation_score(z: pd.DataFrame, z_alert: float) -> np.ndarray:
    """Combine the three largest positive deviations into one 0-1 score."""
    excess = np.clip(np.asarray(z, dtype=float) - z_alert, 0, None)
    if excess.shape[1] < 3:
        excess = np.pad(excess, ((0, 0), (0, 3 - excess.shape[1])))
    top = -np.sort(-excess, axis=1)[:, :3]
    raw = top[:, 0] + 0.5 * top[:, 1] + 0.25 * top[:, 2]
    return 1 - np.exp(-raw / 4.0)


class MLEnsemble:
    def __init__(self, cfg: Config = CONFIG):
        self.cfg = cfg
        self.scaler = StandardScaler()
        self.iforest = IsolationForest(n_estimators=cfg.iforest_trees, random_state=cfg.seed)
        self.lof = LocalOutlierFactor(n_neighbors=cfg.lof_neighbors, novelty=True)
        self.ae = MLPRegressor(hidden_layer_sizes=cfg.autoencoder_layers, activation="tanh",
                               max_iter=800, random_state=cfg.seed, early_stopping=True,
                               validation_fraction=0.15, alpha=1e-3)
        self.calib: dict[str, tuple[float, float]] = {}

    def _raw(self, X: np.ndarray) -> dict:
        Xs = self.scaler.transform(X)
        rec = self.ae.predict(Xs)
        return {"iforest": -self.iforest.score_samples(Xs),
                "lof": -self.lof.score_samples(Xs),
                "autoencoder": np.mean((Xs - rec) ** 2, axis=1)}

    def fit(self, X: np.ndarray) -> "MLEnsemble":
        Xs = self.scaler.fit_transform(X)
        self.iforest.fit(Xs)
        self.lof.fit(Xs)
        self.ae.fit(Xs, Xs)
        for k, s in self._raw(X).items():
            med, q99 = np.median(s), np.quantile(s, 0.99)
            self.calib[k] = (med, max(q99 - med, 1e-9))
        return self

    def score(self, X: np.ndarray) -> pd.DataFrame:
        out = {}
        for k, s in self._raw(X).items():
            med, spread = self.calib[k]
            norm = (s - med) / spread               # 1.0 == 99th percentile of normal behaviour
            out[f"ml_{k}"] = np.clip((norm - 1.0) / 3.0, 0, 1)
            out[f"raw_{k}"] = s
        df = pd.DataFrame(out)
        df["ml"] = df[["ml_iforest", "ml_lof", "ml_autoencoder"]].mean(axis=1)
        return df
```

Threat rules (first three of twenty) (ueba/detectors.py)

```
RULES = [
    Rule("R01", "Brute-force success", 0.80, "T1110 Brute Force", "user",
         lambda d: (d["max_fail_10min"] >= 10) & (d["fail_then_success"] > 0),
         lambda r: f"{r.max_fail_10min:.0f} failed logins within 10 min followed by a successful
login"),
    Rule("R02", "Brute-force / password spray attempt", 0.45, "T1110 Brute Force", "user",
         lambda d: (d["max_fail_10min"] >= 10) & (d["fail_then_success"] == 0),
         lambda r: f"{r.max_fail_10min:.0f} failed logins within 10 minutes"),
    Rule("R03", "Impossible travel", 0.70, "T1078 Valid Accounts", "user",
         lambda d: d["impossible_travel"] > 0,
         lambda r: "Logins from locations too far apart to travel between in the time elapsed"),
    # ... 17 further rules (R04-R15, D01-D05)
]
```

Impossible-travel detection (ueba/features.py)

```
def haversine_km(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, (lat1, lon1, lat2, lon2))
```

```
    a = np.sin((lat2 - lat1) / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) **
2
    return 6371.0 * 2 * np.arcsin(np.sqrt(a))


def impossible_travel_events(auth: pd.DataFrame, max_kmh: float) -> pd.DataFrame:
    """Consecutive successful logins whose implied speed exceeds ``max_kmh``."""
    ok = auth[(auth["result"] == "success") & auth["lat"].notna()].sort_values(["user",
"timestamp"]).copy()
    ok["prev_lat"] = ok.groupby("user")["lat"].shift()
    ok["prev_lon"] = ok.groupby("user")["lon"].shift()
    ok["prev_ts"] = ok.groupby("user")["timestamp"].shift()
    ok["prev_city"] = ok.groupby("user")["city"].shift()
    ok = ok.dropna(subset=["prev_lat"])
    ok["km"] = haversine_km(ok["prev_lat"], ok["prev_lon"], ok["lat"], ok["lon"])
    hrs = (ok["timestamp"] - ok["prev_ts"]).dt.total_seconds() / 3600
    ok["kmh"] = ok["km"] / hrs.clip(lower=1 / 60)
    return ok[(ok["km"] > 500) & (ok["kmh"] > max_kmh)]
```

Analyst feedback (ueba/feedback.py)

```
def record(db_path: Path, entity: str, date: str, verdict: str, note: str = "", analyst: str =
"analyst",
           alert_id: str = "") -> None:
    if verdict not in VERDICTS:
        raise ValueError(f"verdict must be one of {VERDICTS}")
    with sqlite3.connect(db_path) as con:
        _ensure(con)
        con.execute("INSERT OR REPLACE INTO feedback VALUES (?,?,?,?,?,?,?)",
                    (entity, str(date), verdict, note, analyst, alert_id,
                     datetime.now().isoformat(timespec="seconds")))


def as_sets(fb: pd.DataFrame) -> tuple[set, set]:
    """(benign keys, confirmed-threat keys) as sets of (entity, date)."""
    benign = set(zip(fb.loc[fb["verdict"] == "benign", "entity"], fb.loc[fb["verdict"] == "benign",
"date"]))
    threat = set(zip(fb.loc[fb["verdict"] == "threat", "entity"], fb.loc[fb["verdict"] == "threat",
"date"]))
    return benign, threat
```

SUBMISSION PAGE OF THE PAPER

PLAGIARISM REPORT